```
#include "losu.h"
#include "losu_api.h"
#include "losu_vm.h"
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
LosuVm *vm = vm_create(1024);
if (!vm)
{
fprintf(stderr, "Failed to create Losu VM\n");
return 1;
}
if (vm_init(vm, NULL, argc, argv) != 0)
{
fprintf(stderr, "Failed to initialize Losu VM\n");
vm_close(vm);
return 1;
}
if (argc < 2)
{
fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
vm_close(vm);
return 1;
}
const char *filename = argv[1];
if (vm_dofile(vm, filename) != 0)
{
fprintf(stderr, "Failed to execute file: %s\n", filename);
vm_close(vm);
return 1;
}
vm_close(vm);
return 0;
}
#ifndef DEFINE_SOURCE_LOSU_C
#define DEFINE_SOURCE_LOSU_C
#include "losu.h"
#include "losu_errmsg.h"
#include "losu_gc.h"
#include "losu_object.h"
#include "losu_syntax.h"
#include "losu_vm.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
_l_size_t LosuVersionNum = 2 * 255 * 255 + 0 * 255 + 0 * 255;
const char *LosuTypeSystem[] = {
"null",
"number",
```

```
"string",
"function",
"unit",
"pointer",
"coroutine",
"call",
"unknown",
};
const char LosuVersion[] = "2.0.0-beta:" __config_losucore_platform_name;
#ifdef __config_losucore_vm_corepkg
#ifdef __config_losucore_encode_nativeUTF8
#ifdef __config_losucore_encode_winGBK
#error "Invalid encode config"
#endif
#include <unistd.h>
#define toSys(vm, s) (s)
#define toLosu(vm, s) (s)
#define __config_losucore_pkgs_core_sleep(ms) (usleep(ms * 1000))
#endif
#ifdef __config_losucore_encode_winGBK
#ifdef __config_losucore_encode_nativeUTF8
#error "Invalid encode config"
#endif
#include <windows.h>
#define toSys(vm, s) (charset_newObj_toSys(vm, s))
#define toLosu(vm, s) (charset_newObj_toLosu(vm, s))
#define __config_losucore_pkgs_core_sleep(ms) (Sleep(ms))
#endif
static int32_t
__losu_pkgcore_io_print(LosuVm *vm)
{
for (int32_t i = 0; i < arg_num(vm); i++)
fprintf(stdout, "%s", toSys(vm, arg_getstr(vm, i + 1)));
return 0;
}
static int32_t
__losu_pkgcore_io_println(LosuVm *vm)
{
for (int32_t i = 0; i < arg_num(vm); i++)
fprintf(stdout, "%s", toSys(vm, arg_getstr(vm, i + 1)));
fprintf(stdout, "\n");
return 0;
}
static int32_t
__losu_pkgcore_io_input(LosuVm *vm)
{
char text[__config_losucore_vm_iobuff] = {0};
int32_t fmt = arg_gettype(vm, 1);
if (fmt != LosuTypeDefine_null)
{
```

```
if (scanf("%s", text) != 1)
vm_error(vm, "Input failed");
}
else
{
if (fgets(text, __config_losucore_vm_iobuff, stdin) == NULL)
vm_error(vm, "Input overflow");
}
for (int32_t i = 0; text[i]; i++)
if (text[i] == '\n' || text[i] == '\r')
{
text[i] = 0;
break;
}
arg_returnstr(vm, toLosu(vm, text));
return 1;
}
static int32_t
__losu_pkgcore_io_getchar(LosuVm *vm)
{
arg_returnnum(vm, (_l_number)((uint8_t)(fgetc(stdin))));
return 1;
}
static int32_t
__losu_pkgcore_type_type(LosuVm *vm)
{
arg_returnstr(vm, arg_gettypeStr(vm, 1));
return 1;
}
static int32_t
__losu_pkgcore_type_float(LosuVm *vm)
{
arg_returnnum(vm, arg_getnum(vm, 1));
return 1;
}
static int32_t
__losu_pkgcore_type_int(LosuVm *vm)
{
arg_returnnum(vm, (_l_number)((int32_t)arg_getnum(vm, 1)));
return 1;
}
static int32_t
__losu_pkgcore_type_str(LosuVm *vm)
{
arg_returnstr(vm, arg_getstr(vm, 1));
return 1;
}
static int32_t
__losu_pkgcore_type_asc(LosuVm *vm)
{
```

```
arg_returnnum(vm, (_l_number)((uint8_t)(arg_getstr(vm, 1)[0])));
return 1;
}
static int32_t
__losu_pkgcore_type_char(LosuVm *vm)
{
char s[2] = {0};
s[0] = (uint8_t)(arg_getnum(vm, 1));
arg_returnstrlen(vm, s, 1);
return 1;
}
static int32_t
__losu_pkgcore_vm_exec(LosuVm *vm)
{
const char *fn = arg_getstr(vm, 1);
LosuObj *oldtop = vm->top;
int32_t sta = vm_loadfile(vm, toSys(vm, fn));
if (sta)
return 0;
stack_call(vm, 0, -1);
return vm->top - oldtop;
}
static int32_t
__losu_pkgcore_vm_eval(LosuVm *vm)
{
const char *s = arg_getstr(vm, 1);
const char *name = arg_getstr(vm, 2);
LosuObj *oldtop = vm->top;
if (name[0] == '\0')
name = "unknown";
int32_t sta = vm_loadstring(vm, s, name);
if (sta)
return 0;
stack_call(vm, 0, -1);
return vm->top - oldtop;
}
static int32_t
__losu_pkgcore_vm_version(LosuVm *vm)
{
arg_returnstr(vm, LosuVersion);
return 1;
}
static int32_t
__losu_pkgcore_vm_memnow(LosuVm *vm)
{
char p = arg_getstr(vm, 1)[0];
int32_t e = 1;
switch (p)
{
case 'K':
```

```
e = 1024;
break;
case 'M':
e = 1024 * 1024;
break;
default:
break;
}
arg_returnnum(vm, (_l_number)((_l_number)gc_getmemNow(vm) / (_l_number)e));
return 1;
}
static int32_t
__losu_pkgcore_vm_memmax(LosuVm *vm)
{
char p = arg_getstr(vm, 1)[0];
int32_t e = 1;
switch (p)
{
case 'K':
e = 1024;
break;
case 'M':
e = 1024 * 1024;
break;
default:
break;
}
arg_returnnum(vm, (_l_number)((_l_number)gc_getmemMax(vm) / (_l_number)e));
return 1;
}
static int32_t
__losu_pkgcore_vm_setgcmax(LosuVm *vm)
{
_l_number i = arg_getnum(vm, 1);
char p = arg_getstr(vm, 2)[0];
int32_t e = 1;
switch (p)
{
case 'K':
e = 1024;
break;
case 'M':
e = 1024 * 1024;
break;
default:
break;
}
gc_setmax(vm, (_l_gcint)(i * e));
return 0;
}
```

```
static int32_t
__losu_pkgcore_except_try(LosuVm *vm)
{
LosuObj *oldtop = vm->top;
LosuObj *oldbase = vm->base;
__losuvmJmp *oldjmp = vm->errjmp;
LosuObj *cfunc = arg_get(vm, 1), *pfunc = arg_get(vm, 2);
if (ovtype(cfunc) != LosuTypeDefine_function || ovtype(pfunc) !=
LosuTypeDefine_function)
vm_error(vm, "try() need 'function' type args");
__losuvmJmp njmp = {
.func = pfunc,
};
vm->errjmp = &njmp;
int32_t sig = setjmp(njmp.jmpflag);
switch (sig)
{
case VMSIGOK:
{
stack_push(vm, *cfunc);
stack_call(vm, 0, 0);
vm->top = oldtop;
vm->base = oldbase;
vm->errjmp = oldjmp;
return 0;
}
case VMSIGERR:
{
vm->top = oldtop;
vm->base = oldbase;
vm->errjmp = oldjmp;
arg_returnnum(vm, VMSIGERR);
return 1;
}
case VMSIGYIELD:
{
vm->top = oldtop;
vm->base = oldbase;
vm->errjmp = oldjmp;
arg_returnnum(vm, VMSIGYIELD);
stack_yield(vm, 1);
break;
}
case VMSIGKILL:
{
vm->errjmp = oldjmp;
__losu_sigThrow(vm, VMSIGKILL);
break;
}
default:
```

```
break;
}
return 0;
}
static int32_t
__losu_pkgcore_except_assert(LosuVm *vm)
{
int32_t narg = arg_num(vm);
int32_t bool = arg_gettype(vm, 1);
if (bool != LosuTypeDefine_null)
return 0;
if (vm->errjmp->func)
{
stack_push(vm, *vm->errjmp->func);
for (int32_t i = 2; i <= narg; i++)
stack_push(vm, *arg_get(vm, i));
stack_call(vm, narg - 1, 0);
__losu_sigThrow(vm, VMSIGERR);
return 0;
}
else
vm_error(vm, toSys(vm, arg_getstr(vm, 2)));
return 0;
}
static int32_t
__losu_pkgcore_except_except(LosuVm *vm)
{
fprintf(stderr, "CORE PANIC, __THROW '%s'", toSys(vm, arg_getstr(vm, 1)));
__losu_sigThrow(vm, VMSIGKILL);
return 0;
}
static int32_t
__losu_pkgcore_asy_async(LosuVm *vm)
{
LosuObj *func = arg_get(vm, 1);
int32_t nargs = arg_num(vm) - 1;
LosuObj *args = arg_get(vm, 2);
if (ovtype(func) != LosuTypeDefine_function)
return 0;
arg_return(vm, obj_newcoro(vm, func, nargs, args));
return 1;
}
static int32_t
__losu_pkgcore_asy_await(LosuVm *vm)
{
uint32_t rt = 0;
for (int32_t i = 0; i < arg_num(vm); i++)
rt += obj_awaitcoro(vm, arg_get(vm, i + 1));
arg_returnnum(vm, rt);
return rt > 0 ? 1 : 0;
```

```
}
static int32_t
__losu_pkgcore_module_import(LosuVm *vm)
{
int32_t narg = arg_num(vm);
const char *pkname = arg_getstr(vm, 1);
LosuPackage_t *pkg = NULL;
for (int32_t i = 0; _inlinePackage[i].name; i++)
if (strcmp(_inlinePackage[i].name, pkname) == 0)
{
pkg = &_inlinePackage[i];
break;
}
if (!pkg)
return 0;
LosuModule_t *mod = pkg->loadpackage(vm);
if (narg == 1)
{
for (int32_t i = 0; mod[i].name; i++)
mod[i].loadmod(vm);
}
else
{
for (int32_t i = 2; i <= narg; i++)
{
char *mname = (char *)arg_getstr(vm, i);
for (int32_t j = 0; mod[j].name; j++)
if (strcmp(mname, mod[j].name) == 0)
{
mod[j].loadmod(vm);
break;
}
}
}
return 0;
}
static int32_t
__losu_pkgcore_module_new(LosuVm *vm)
{
int32_t args = arg_num(vm);
LosuObj *class = arg_get(vm, 1);
LosuObj *func = (LosuObj *)&_inlineNullObj;
LosuObj newobj = obj_newunit(vm);
if (obj_type(vm, (class)) != LosuTypeDefine_unit)
vm_error(vm, __config_losucore_errmsg_msgInvalidNewobj);
{
LosuNode *p = obj_unit_first(vm, *class);
while (p != NULL)
{
obj_setunit(vm, newobj, obj_unit_nodekey(vm, p),
```

```
obj_unit_nodevalue(vm, p));
p = obj_unit_next(vm, *class, p);
}
}
func = obj_indexunitbystr(vm, newobj, "__init__");
if (obj_type(vm, func) == LosuTypeDefine_function)
{
stack_push(vm, *func);
stack_push(vm, newobj);
for (int32_t i = 2; i <= args; i++)
stack_push(vm, *arg_get(vm, i));
stack_call(vm, args, 0);
}
arg_return(vm, newobj);
return 1;
}
static int32_t
__losu_pkgcore_sys_system(LosuVm *vm)
{
arg_returnnum(vm, system(toSys(vm, arg_getstr(vm, 1))));
return 1;
}
static int32_t
__losu_pkgcore_sys_getenv(LosuVm *vm)
{
const char *n = arg_getstr(vm, 1);
char *v = getenv(toSys(vm, n));
if (!v)
return 0;
arg_returnstr(vm, toLosu(vm, v));
return 1;
}
extern int32_t setenv(const char *n, const char *v, int owrite);
static int32_t
__losu_pkgcore_sys_setenv(LosuVm *vm)
{
const char *n = arg_getstr(vm, 1);
const char *v = arg_getstr(vm, 2);
setenv(toSys(vm, n), toSys(vm, v), 1);
return 0;
}
static int32_t __losu_pkgcore_math_abs(LosuVm *vm)
{
_l_number num = arg_getnum(vm, 1);
arg_returnnum(vm, num < 0 ? -num : num);
return 1;
}
static int32_t
__loadcorepkg__(LosuVm *vm)
{
```

```
struct
{
const char *n;
LosuApi f;
} funcs[] = {
{"print", __losu_pkgcore_io_print},
{"println", __losu_pkgcore_io_println},
{"abs", __losu_pkgcore_math_abs},
{"input", __losu_pkgcore_io_input},
{"getchar", __losu_pkgcore_io_getchar},
{"type", __losu_pkgcore_type_type},
{"float", __losu_pkgcore_type_float},
{"int", __losu_pkgcore_type_int},
{"str", __losu_pkgcore_type_str},
{"asc", __losu_pkgcore_type_asc},
{"char", __losu_pkgcore_type_char},
{"exec", __losu_pkgcore_vm_exec},
{"eval", __losu_pkgcore_vm_eval},
{"version", __losu_pkgcore_vm_version},
{"memnow", __losu_pkgcore_vm_memnow},
{"memmax", __losu_pkgcore_vm_memmax},
{"setgcmax", __losu_pkgcore_vm_setgcmax},
{"try", __losu_pkgcore_except_try},
{"assert", __losu_pkgcore_except_assert},
{"except", __losu_pkgcore_except_except},
{"async", __losu_pkgcore_asy_async},
{"await", __losu_pkgcore_asy_await},
{"import", __losu_pkgcore_module_import},
{"new", __losu_pkgcore_module_new},
{"system", __losu_pkgcore_sys_system},
{"getenv", __losu_pkgcore_sys_getenv},
{"setenv", __losu_pkgcore_sys_setenv},
};
for (int32_t i = 0; i < sizeof(funcs) / sizeof(funcs[0]); i++)
vm_setval(vm, funcs[i].n, obj_newfunction(vm, funcs[i].f));
return 0;
}
#endif
LosuPackage_t *_inlinePackage = NULL;
int32_t
vm_init(LosuVm *vm, LosuPackage_t *pkgs_t, int32_t argc, const char **argv)
{
LosuObj o = obj_newunit(vm);
obj_setunitbystr(vm, o, "num", obj_newnum(vm, argc - 1));
obj_setunitbystr(vm, o, "self",
obj_newstr(vm, charset_newObj_toLosu(vm, argv[0])));
for (int32_t i = 1; i < argc; i++)
obj_setunitbynum(vm, o, i,
obj_newstr(vm, charset_newObj_toLosu(vm, argv[i])));
vm_setval(vm, "args", o);
```

```
#ifdef __config_losucore_vm_corepkg
__loadcorepkg__(vm);
#endif
_inlinePackage = pkgs_t;
gc_setmax(vm, __config_losucore_gc_defaultsize);
return 0;
}
#endif
#ifndef DEFINE_SOURCE_LOSU_API_C
#define DEFINE_SOURCE_LOSU_API_C
#include "losu_api.h"
#include "losu_bytecode.h"
#include "losu_errmsg.h"
#include "losu_gc.h"
#include "losu_malloc.h"
#include "losu_object.h"
#include "losu_syntax.h"
#include "losu_vm.h"
#include <setjmp.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifndef DEFINE_SOURCE_LOSU_API_C_CAPI
#define DEFINE_SOURCE_LOSU_API_C_CAPI
LosuExtern LosuVm *
vm_create (int32_t size)
{
__losuvmJmp lj = {
.func = NULL,
};
if (size <= 0)
return NULL;
LosuVm *vm = __losu_mem_new (NULL, LosuVm);
if (vm == NULL)
return NULL;
*vm = (LosuVm){
.stack = NULL,
.top = NULL,
.stackmax = NULL,
.base = NULL,
.mstack = NULL,
.mtop = NULL,
.mstackmax = NULL,
.errjmp = &lj,
.bufftmp = NULL,
.nbufftmp = 0,
.inspool = NULL,
.funcpool = NULL,
.hashpool = NULL,
```

```
.strpool = (__losuvmStrseg){
.size = 0,
.nsize = 0,
.strobj = NULL,
},
.callpool = NULL,
.coropool = NULL,
.nblocks = sizeof(LosuVm),
.gcDymax = 1,
.gcMax = SIZE_MAX,
.gcHook = 0,
.name = NULL,
};
int32_t jsta = setjmp (lj.jmpflag);
switch (jsta)
{
case VMSIGOK:
{
vm->global = __losu_objUnit_new (vm, 0);
__losu_vmHeap_init (vm, size);
__losu_objStringPool_init (vm);
__losu_syntaxLex_init (vm);
break;
}
case VMSIGERR:
{
vm_close (vm);
return NULL;
break;
}
default:
break;
}
vm->errjmp = NULL;
return vm;
}
LosuExtern int vm_init (LosuVm *vm, LosuPackage_t *pkgs_t, int32_t argc,
const char **argv);
LosuExtern void
vm_stop (LosuVm *vm, const char *estr, ...)
{
va_list ap;
va_start (ap, estr);
vsprintf ((char *)(vm->staticbuff), estr, ap);
fprintf (stderr, "Core Panic: %s\nAt line %d of '%1s'\n",
(char *)vm->staticbuff, __losuApigetline (vm, vm->top - 1),
vm->name != NULL ? vm->name : "<unknown>");
va_end (ap);
__losu_sigThrow (vm, VMSIGKILL);
}
```

```
LosuExtern void
vm_error (LosuVm *vm, const char *estr, ...)
{
va_list ap;
va_start (ap, estr);
vsprintf ((char *)(vm->staticbuff), estr, ap);
fprintf (stderr, "Runtime Error: %s\nAt line %d of '%ls'\n",
(char *)vm->staticbuff, __losuApigetline (vm, vm->top - 1),
vm->name != NULL ? vm->name : "<unknown>");
va_end (ap);
__losu_sigThrow (vm, VMSIGERR);
}
LosuExtern int32_t
vm_dofile (LosuVm *vm, const char *f)
{
int32_t i;
if (!(i = vm_loadfile (vm, f)))
return vm_execute (vm, 0, -1, f);
return i;
}
LosuExtern int32_t
vm_dostring (LosuVm *vm, const char *s, const char *n)
{
int32_t i;
if (!(i = vm_loadstring (vm, s, n)))
return vm_execute (vm, 0, -1, n);
return i;
}
LosuExtern int32_t
vm_dobyte (LosuVm *vm, const char *byte, size_t len, const char *name)
{
int32_t i;
if (!(i = vm_loadbyte (vm, byte, len, name)))
return vm_execute (vm, 0, -1, name);
return i;
}
int32_t
vm_loadfile (LosuVm *vm, const char *f)
{
return __losu_syntaxIO_loadfile (vm, f);
}
LosuExtern int32_t
vm_loadstring (LosuVm *vm, const char *s, const char *name)
{
return __losu_syntaxIO_loadstring (vm, s, strlen (s), name);
}
LosuExtern int32_t
vm_loadbyte (LosuVm *vm, const char *byte, size_t len, const char *name)
{
return __losu_syntaxIO_loadstring (vm, byte, len, name);
```

```
}
int32_t
vm_execute (LosuVm *vm, int32_t narg, int32_t nres, const char *name)
{
int32_t i = 0;
{
LosuObj *func = (vm->top - 1) - narg;
LosuObj *oldbase = vm->base;
LosuObj *oldtop = vm->top;
__losuvmJmp *oldjmp = vm->errjmp;
const char *oldname = vm->name;
__losuvmJmp jmp = {
.func = NULL,
};
vm->name = name;
vm->errjmp = &jmp;
int32_t jsta = setjmp (jmp.jmpflag);
switch (jsta)
{
case VMSIGOK:
{
__losu_vmHeap_rawcall (vm, func, nres);
break;
}
case VMSIGERR:
{
vm->base = oldbase;
vm->top = oldtop;
vm->name = oldname;
vm->errjmp = oldjmp;
break;
}
case VMSIGYIELD:
{
vm->base = oldbase;
vm->top = oldtop;
vm->errjmp = oldjmp;
__losu_sigThrow (vm, VMSIGYIELD);
vm->name = oldname;
break;
}
case VMSIGKILL:
{
vm->errjmp = oldjmp;
__losu_sigThrow (vm, VMSIGKILL);
vm->name = oldname;
break;
}
default:
break;
```

```
}
i = jsta;
}
return i;
}
LosuExtern LosuObj *
vm_getval (LosuVm *vm, const char *name)
{
return (LosuObj *)__losu_objUnit_getstr (vm->global,
__losu_objString_new (vm, name));
}
LosuExtern void
vm_setval (LosuVm *vm, const char *name, LosuObj val)
{
LosuObj key = (LosuObj){
.type = LosuTypeDefine_string,
.value.str = __losu_objString_new (vm, name),
};
*__losu_objUnit_set (vm, vm->global, &key) = val;
}
LosuExtern void
vm_close (LosuVm *vm)
{
__losu_gc_collect (vm, 1);
__losu_objStringPool_deinit (vm);
if (vm->mstack)
{
__losu_mem_free (vm, vm->mstack);
vm->nblocks -= (vm->mstackmax - vm->mstack + 1) * sizeof (LosuObj);
}
if (vm->bufftmp)
{
__losu_mem_free (vm, vm->bufftmp);
vm->nblocks -= vm->nbufftmp * sizeof (char);
}
__losu_mem_free (vm, vm);
}
LosuExtern void
gc_setmax (LosuVm *vm, _l_gcint size)
{
vm->gcMax = vm->gcDymax = size;
}
LosuExtern _l_gcint
gc_getmemNow (LosuVm *vm)
{
return vm->nblocks;
}
LosuExtern _l_gcint
gc_getmemMax (LosuVm *vm)
{
```

```
return vm->gcHook;
}
LosuExtern _l_bool
gc_collect (LosuVm *vm)
{
return __losu_gc_checkClt (vm);
}
LosuExtern int32_t
arg_num (LosuVm *vm)
{
return (int32_t)(vm->top - vm->base);
}
LosuExtern int
arg_gettype (LosuVm *vm, int idx)
{
return obj_type (vm, arg_get (vm, idx));
}
LosuExtern const char *
arg_gettypeStr (LosuVm *vm, int idx)
{
return obj_typeStr (vm, arg_get (vm, idx));
}
LosuExtern LosuObj *
arg_get (LosuVm *vm, int idx)
{
if (idx > 0)
{
LosuObj *o = vm->base + (idx - 1);
if (o >= vm->top)
ovtype (o) = LosuTypeDefine_null;
return o;
}
return (LosuObj *)(&_inlineNullObj);
}
LosuExtern _l_number
arg_getnum (LosuVm *vm, int idx)
{
return obj_tonum (vm, arg_get (vm, idx));
}
LosuExtern const char *
arg_getstr (LosuVm *vm, int idx)
{
return obj_tostr (vm, arg_get (vm, idx));
}
LosuExtern LosuApi
arg_getfunc (LosuVm *vm, int idx)
{
return obj_tofunction (vm, arg_get (vm, idx));
}
LosuExtern char *
```

```
arg_getptr (LosuVm *vm, int idx)
{
return obj_toptr (vm, arg_get (vm, idx));
}
LosuExtern void
arg_return (LosuVm *vm, LosuObj obj)
{
*(vm->top) = (obj);
if (vm->top == vm->stackmax)
vm_error (vm, __config_losucore_errmsg_msgStackOverflow);
vm->top++;
}
LosuExtern void
arg_returnnull (LosuVm *vm)
{
arg_return (vm, obj_newnull (vm));
}
LosuExtern void
arg_returnnum (LosuVm *vm, _l_number num)
{
arg_return (vm, obj_newnum (vm, num));
}
LosuExtern void
arg_returnstr (LosuVm *vm, const char *str)
{
arg_return (vm, obj_newstr (vm, (char *)str));
}
LosuExtern void
arg_returnstrlen (LosuVm *vm, const char *str, size_t len)
{
arg_return (vm, obj_newstrlen (vm, (char *)str, len));
}
LosuExtern void
arg_returnfunc (LosuVm *vm, LosuApi func)
{
arg_return (vm, obj_newfunction (vm, func));
}
LosuExtern void
arg_returnptr (LosuVm *vm, char *ptr)
{
arg_return (vm, obj_newptr (vm, ptr));
}
LosuExtern int
obj_type (LosuVm *vm, LosuObj *obj)
{
return ovtype (obj);
}
LosuExtern const char *
obj_typeStr (LosuVm *vm, LosuObj *obj)
{
```

```
if (ovtype (obj) < LosuTypeDefine_unknown)
{
if (ovtype (obj) == LosuTypeDefine_unit)
{
LosuObj *o = obj_indexunitbystr (vm, *obj, "__type__");
if (ovtype (o) != LosuTypeDefine_null)
return obj_tostr (vm, o);
}
return LosuTypeSystem[ovtype (obj)];
}
else
return LosuTypeSystem[LosuTypeDefine_unknown];
}
LosuExtern LosuObj
obj_newnull (LosuVm *vm)
{
return (LosuObj){
.type = LosuTypeDefine_null,
};
}
LosuExtern LosuObj
obj_newnum (LosuVm *vm, _l_number num)
{
return (LosuObj){
.type = LosuTypeDefine_number,
.value.num = num,
};
}
LosuExtern _l_number
obj_tonum (LosuVm *vm, LosuObj *obj)
{
#define tonumber(o)
\
((ovtype (o) != LosuTypeDefine_number)                                    \
&& (__losu_vmCore_Tonum (vm, o, 0)) != 0)
return (obj == NULL || tonumber (obj)) ? 0 : ovnumber (obj);
#undef tonumber
}
LosuExtern LosuObj
obj_newstr (LosuVm *vm, char *str)
{
return (LosuObj){
.type = LosuTypeDefine_string,
.value.str = __losu_objString_new (vm, str),
};
}
LosuExtern LosuObj
obj_newstrlen (LosuVm *vm, char *str, size_t len)
{
return (LosuObj){
```

```
.type = LosuTypeDefine_string,
.value.str = __losu_objString_newstr (vm, str, len),
};
}
LosuExtern size_t
obj_getstrlen (LosuVm *vm, LosuObj *obj)
{
return (ovtype (obj) == LosuTypeDefine_string ? ovIstr (obj)->len : 0);
}
LosuExtern const char *
obj_tostr (LosuVm *vm, LosuObj *obj)
{
#define tostring(o)
\
((ovtype (o) != LosuTypeDefine_string)                                    \
&& (__losu_vmCore_Tostr (vm, o, 0) != 0))
return (obj == NULL || tostring (obj)) ? "" : ovSstr (obj);
#undef tostring
}
LosuExtern LosuObj
obj_newfunction (LosuVm *vm, LosuApi func)
{
_inlineFunction *f = __losu_objFunc_new (vm, 0);
f->isC = 1;
f->func.capi = func;
return (LosuObj){
.type = LosuTypeDefine_function,
.value.func = f,
};
}
LosuExtern LosuApi
obj_tofunction (LosuVm *vm, LosuObj *obj)
{
return (ovtype (obj) == LosuTypeDefine_function
? (ovfunc (obj)->isC ? ovfunc (obj)->func.capi : NULL)
: NULL);
}
LosuExtern LosuObj
obj_newunit (LosuVm *vm)
{
return (LosuObj){
.type = LosuTypeDefine_unit,
.value.hash = __losu_objUnit_new (vm, 0),
};
}
LosuExtern LosuObj *
obj_indexunit (LosuVm *vm, LosuObj unit, LosuObj key)
{
if (unit.type == LosuTypeDefine_unit)
return (LosuObj *)__losu_objUnit_get (ovhash ((&unit)), &key);
```

```
return (LosuObj *)&_inlineNullObj;
}
LosuExtern LosuObj *
obj_indexunitbynum (LosuVm *vm, LosuObj unit, _l_number i)
{
if (unit.type == LosuTypeDefine_unit)
return (LosuObj *)__losu_objUnit_getnum (ovhash ((&unit)), i);
return (LosuObj *)&_inlineNullObj;
}
LosuExtern LosuObj *
obj_indexunitbystr (LosuVm *vm, LosuObj unit, char *s)
{
LosuObj o = obj_newstr (vm, s);
if (unit.type == LosuTypeDefine_unit)
return (LosuObj *)__losu_objUnit_getstr (ovhash ((&unit)), ovIstr ((&o)));
return (LosuObj *)&_inlineNullObj;
}
LosuExtern void
obj_setunit (LosuVm *vm, LosuObj unit, LosuObj key, LosuObj value)
{
if (unit.type == LosuTypeDefine_unit)
*__losu_objUnit_set (vm, ovhash ((&unit)), &key) = value;
}
LosuExtern void
obj_setunitbynum (LosuVm *vm, LosuObj unit, _l_number key, LosuObj value)
{
if (unit.type == LosuTypeDefine_unit)
*__losu_objUnit_setnum (vm, ovhash ((&unit)), key) = value;
}
LosuExtern void
obj_setunitbystr (LosuVm *vm, LosuObj unit, char *key, LosuObj value)
{
if (unit.type == LosuTypeDefine_unit)
*__losu_objUnit_setstr (vm, ovhash ((&unit)),
__losu_objString_new (vm, key))
= value;
}
LosuExtern LosuNode *
obj_unit_first (LosuVm *vm, LosuObj unit)
{
if (ovtype (&unit) == LosuTypeDefine_unit)
return (LosuNode *)__losu_objUnit_getnext (ovhash ((&unit)),
(LosuObj *)&_inlineNullObj);
return NULL;
}
LosuExtern LosuNode
obj_unit_location (LosuVm *vm, LosuObj unit, LosuObj key)
{
LosuNode n;
n.key = key;
```

```
if (unit.type == LosuTypeDefine_unit)
n.value = *__losu_objUnit_get (ovhash ((&unit)), &key);
else
n.value = _inlineNullObj;
return n;
}
LosuExtern LosuNode *
obj_unit_next (LosuVm *vm, LosuObj unit, LosuNode *n)
{
if (unit.type == LosuTypeDefine_unit)
return (LosuNode *)__losu_objUnit_getnext (ovhash ((&unit)), (&(n->key)));
return NULL;
}
LosuExtern LosuObj
obj_unit_nodekey (LosuVm *vm, LosuNode *n)
{
return n->key;
}
LosuExtern LosuObj
obj_unit_nodevalue (LosuVm *vm, LosuNode *n)
{
return n->value;
}
LosuExtern LosuObj
obj_newptr (LosuVm *vm, char *ptr)
{
return (LosuObj){
.type = LosuTypeDefine_ptr,
.value.ptr = ptr,
};
}
LosuExtern char *
obj_toptr (LosuVm *vm, LosuObj *obj)
{
return (ovtype (obj) == LosuTypeDefine_ptr ? obj->value.ptr : NULL);
}
LosuExtern LosuObj
obj_newcoro (LosuVm *vm, LosuObj *func, int32_t nargs, LosuObj *args)
{
_inlineCoroutine *newcoro = __losu_objCoroutine_new (vm);
*(newcoro)->stack = *func;
for (int32_t i = 1; i <= nargs; i++)
*(newcoro->stack + i) = args[i - 1];
newcoro->top = newcoro->stack + nargs + 1;
return (LosuObj){
.type = LosuTypeDefine_coroutine,
.value.coro = newcoro,
};
}
LosuExtern _l_bool
```

```
obj_awaitcoro (LosuVm *vm, LosuObj *coro)
{
_l_bool rt = 0;
LosuObj *oldtop = vm->top;
LosuObj *oldbase = vm->base;
LosuObj *oldstack = vm->stack;
LosuObj *oldstackmax = vm->stackmax;
if (ovtype (coro) != LosuTypeDefine_coroutine)
return 0;
__losuvmJmp corojmp={
.func = NULL,
}, *oldjmp = vm->errjmp;
vm->errjmp = &corojmp;
vm->stack = vm->base = ovcoro (coro)->stack;
vm->top = ovcoro (coro)->top;
vm->stackmax = vm->stack + __config_losucore_vm_corosize - 1;
int32_t jid = setjmp (corojmp.jmpflag);
switch (jid)
{
case VMSIGOK:
{
if (ovcoro (coro)->sta == LosuCoroStateREADY)
{
ovcoro (coro)->sta = LosuCoroStateRUN;
__losu_vmHeap_procall (vm, vm->stack, 0);
ovcoro (coro)->sta = LosuCoroStateEND;
rt = 1;
break;
}
if (ovcoro (coro)->sta == LosuCoroStateYIELD)
{
LosuObj *recall = NULL;
LosuObj *tmp = recall;
for (LosuObj *o = vm->stack; o < vm->top; o++)
if (ovtype (o) == LosuTypeDefine_callinfo)
{
if (ovcall (o)->func->isC
|| !ovcall (o)->isMalloc)
{
vm->top = o;
__losu_sigThrow (vm, VMSIGKILL);
break;
}
if (recall == NULL)
{
recall = tmp = o;
ovcall (o)->nextobj = NULL;
}
else
{
```

```
ovcall (tmp)->nextobj = o;
tmp = o;
ovcall (tmp)->nextobj = NULL;
}
}
ovcoro (coro)->sta = LosuCoroStateRUN;
__losu_vmCore_exec (vm, NULL, recall);
ovcoro (coro)->sta = LosuCoroStateEND;
rt = 1;
break;
}
break;
}
case VMSIGERR:
{
ovcoro (coro)->sta = LosuCoroStateEND;
rt = 0;
break;
}
case VMSIGYIELD:
{
ovcoro (coro)->sta = LosuCoroStateYIELD;
rt = 1;
ovcoro (coro)->top = vm->top;
break;
}
case VMSIGKILL:
{
vm->errjmp = oldjmp;
__losu_sigThrow (vm, VMSIGKILL);
}
default:
break;
}
vm->stack = oldstack;
vm->base = oldbase;
vm->top = oldtop;
vm->stackmax = oldstackmax;
vm->errjmp = oldjmp;
return rt;
}
LosuExtern void
stack_push (LosuVm *vm, LosuObj o)
{
*(vm->top) = o;
if (vm->top == vm->stackmax)
vm_error (vm, __config_losucore_errmsg_msgStackOverflow);
vm->top++;
}
LosuExtern void
```

```
stack_pop (LosuVm *vm, int i)
{
vm->top -= i;
if (vm->top < vm->stack)
vm->top = vm->stack;
}
LosuExtern void
stack_procall (LosuVm *vm, int argnum, int resnum)
{
__losu_vmHeap_procall (vm, vm->top - (argnum + 1), resnum);
}
LosuExtern void
stack_call (LosuVm *vm, int32_t argnum, int32_t resnum)
{
__losu_vmHeap_rawcall (vm, vm->top - (argnum + 1), resnum);
}
LosuExtern void
stack_yield (LosuVm *vm, int32_t res)
{
if (vm->stack == vm->mstack)
return;
LosuObj *func = vm->base - 1;
LosuObj *fres = vm->top - res;
int32_t nres = ovcall (func)->nres;
if (nres == -1)
{
while (fres < vm->top)
*func++ = *fres++;
vm->top = func;
}
else
{
for (; nres > 0 && fres < vm->top; nres--)
*func++ = *fres++;
vm->top = func;
for (; nres > 0; nres--)
ovtype ((vm->top++)) = LosuTypeDefine_null;
}
__losu_sigThrow (vm, VMSIGYIELD);
}
#ifdef __config_losucore_encode_winGBK
#include <windows.h>
#endif
LosuExtern char *
charset_toSys (const char *s)
{
#ifdef __config_losucore_encode_nativeUTF8
return (char *)s;
#endif
#ifdef __config_losucore_encode_winGBK
```

```
return __losuWintoGBK (s);
#endif
}
LosuExtern char *
charset_toLosu (const char *s)
{
#ifdef __config_losucore_encode_nativeUTF8
return (char *)s;
#endif
#ifdef __config_losucore_encode_winGBK
return __losuWintoUTF8 (s);
#endif
}
LosuExtern char *
charset_newObj_toLosu (LosuVm *vm, const char *s)
{
char *t = charset_toLosu (s);
if (t == NULL)
return NULL;
LosuObj o = obj_newstr (vm, t);
if (t != s)
__losu_mem_free (NULL, t);
return ovSstr ((&o));
}
LosuExtern char *
charset_newObj_toSys (LosuVm *vm, const char *s)
{
char *t = charset_toSys (s);
if (t == NULL)
return NULL;
LosuObj o = obj_newstr (vm, t);
if (t != s)
__losu_mem_free (NULL, t);
return ovSstr ((&o));
}
#endif
#ifndef DEFINE_SOURCE_LOSU_API_C_STATIC
#define DEFINE_SOURCE_LOSU_API_C_STATIC
#ifdef __config_losucore_encode_winGBK
#include <wchar.h>
#include <windows.h>
static char *
__losuWintoGBK (const char *str)
{
int len = MultiByteToWideChar (CP_UTF8, 0, str, -1, NULL, 0);
wchar_t *strUnicode
= (wchar_t *)__losu_mem_malloc (NULL, len * sizeof (wchar_t *));
wmemset (strUnicode, 0, len);
MultiByteToWideChar (CP_UTF8, 0, str, -1, strUnicode, len);
len = WideCharToMultiByte (CP_ACP, 0, strUnicode, -1, NULL, 0, NULL, NULL);
```

```
char *strGbk = (char *)__losu_mem_malloc (NULL, len * sizeof (char *));
memset (strGbk, 0, len);
WideCharToMultiByte (CP_ACP, 0, strUnicode, -1, strGbk, len, NULL, NULL);
__losu_mem_free (NULL, strUnicode);
strUnicode = NULL;
return strGbk;
}
static char *
__losuWintoUTF8 (const char *str)
{
int len = MultiByteToWideChar (CP_ACP, 0, str, -1, NULL, 0);
wchar_t *strUnicode
= (wchar_t *)__losu_mem_malloc (NULL, len * sizeof (wchar_t *));
wmemset (strUnicode, 0, len);
MultiByteToWideChar (CP_ACP, 0, str, -1, strUnicode, len);
len = WideCharToMultiByte (CP_UTF8, 0, strUnicode, -1, NULL, 0, NULL, NULL);
char *strutf8 = (char *)__losu_mem_malloc (NULL, len * sizeof (char *));
memset (strutf8, 0, len);
WideCharToMultiByte (CP_UTF8, 0, strUnicode, -1, strutf8, len, NULL, NULL);
__losu_mem_free (NULL, strUnicode);
strUnicode = NULL;
return strutf8;
}
#endif
static int32_t
__losuApigetline (LosuVm *vm, LosuObj *obj)
{
while (1)
{
if (!(obj && ovtype (obj) == LosuTypeDefine_callinfo
&& !ovcall (obj)->func->isC))
obj--;
else
break;
if (obj < vm->stack)
return -1;
}
return __losuApitryline (ovcall (obj)->func->func.sdef->lineinfo,
__losuApicurrentpc (obj));
}
static int32_t
__losuApicurrentpc (LosuObj *obj)
{
if (ovcall (obj)->pc)
return ((ovcall (obj)->pc) - ovcall (obj)->func->func.sdef->code) - 1;
else
return -1;
}
static int32_t
__losuApitryline (int32_t *lineinfo, int32_t pc)
```

```
{
int32_t refi = 0;
int32_t refline = 1;
if (lineinfo == NULL || pc == -1)
return -1;
if (lineinfo[refi] < 0)
refline += -lineinfo[refi++];
while (lineinfo[refi] > pc)
{
refline--;
refi--;
if (lineinfo[refi] < 0)
refline -= -lineinfo[refi++];
}
while (1)
{
int32_t nextline = refline + 1;
int32_t nexref = refi + 1;
if (lineinfo[nexref] < 0)
nextline += -lineinfo[nexref++];
if (lineinfo[nexref] > pc)
break;
refline = nextline;
refi = nexref;
}
return refline;
}
#endif
#endif
#ifndef DEFINE_SOURCE_LOSU_GC_C
#define DEFINE_SOURCE_LOSU_GC_C
#include "losu_gc.h"
#include "losu.h"
#include "losu_errmsg.h"
#include "losu_malloc.h"
#include "losu_object.h"
#include <stdio.h>
#ifndef DEFINE_SOURCE_LOSU_GC_C_STATICDEC
#define DEFINE_SOURCE_LOSU_GC_C_STATICDEC
static void __losuGcMark (LosuVm *vm);
static void __losuGcmarkObj (_inlineGc *gc, LosuObj *o);
static void __losuGcmarkStack (LosuVm *vm, _inlineGc *gc);
static void __losuGcmarkFunc (_inlineGc *gc, _inlineFunction *f);
static void __losuGcmarkScode (_inlineScode *f);
static void __losuGcCollect (LosuVm *vm);
static void __losuGccollectScode (LosuVm *vm);
static void __losuGccollectFunc (LosuVm *vm);
static void __losuGccollectUnit (LosuVm *vm);
static void __losuGccollectString (LosuVm *vm, _l_bool all);
static void __losuGccollectCallinfo (LosuVm *vm);
```

```
static void __losuGccollectCoroutine (LosuVm *vm);
static void __losuGcAdjbuff (LosuVm *vm);
void
__losu_gc_collect (LosuVm *vm, _l_bool all)
{
__losuGccollectString (vm, all);
__losuGccollectUnit (vm);
__losuGccollectScode (vm);
__losuGccollectFunc (vm);
__losuGccollectCallinfo (vm);
__losuGccollectCoroutine (vm);
__losuGcAdjbuff (vm);
}
_l_bool
__losu_gc_checkClt (LosuVm *vm)
{
if (vm->nblocks > vm->gcHook)
vm->gcHook = vm->nblocks;
if (vm->nblocks >= vm->gcDymax)
{
__losuGcCollect (vm);
if (vm->gcDymax > vm->gcMax)
vm->gcDymax = vm->gcMax;
return 1;
}
return 0;
}
#endif
#ifndef DEFINE_SOURCE_LOSU_GC_C_GCMARK
#define DEFINE_SOURCE_LOSU_GC_C_GCMARK
static void
__losuGcMark (LosuVm *vm)
{
_inlineGc gc;
gc.func = NULL;
gc.unit = vm->global;
vm->global->mark = NULL;
__losuGcmarkStack (vm, &gc);
while (1)
{
if (gc.func)
{
_inlineFunction *f = gc.func;
gc.func = f->mark;
for (int32_t i = 0; i < f->nclosure; i++)
__losuGcmarkObj (&gc, &f->closure[i]);
}
else if (gc.unit)
{
_inlineHash *h = gc.unit;
```

```
gc.unit = h->mark;
for (int32_t i = 0; i < h->size; i++)
{
LosuNode *n = &h->node[i];
if (ovtype ((&n->key)) != LosuTypeDefine_null)
{
if (ovtype ((&n->value)) == LosuTypeDefine_null)
{
ovtype ((&n->key)) = LosuTypeDefine_number;
ovnumber ((&n->key)) = 0;
}
__losuGcmarkObj (&gc, &n->key);
__losuGcmarkObj (&gc, &n->value);
}
}
}
else
break;
}
}
static void
__losuGcmarkObj (_inlineGc *gc, LosuObj *o)
{
switch (ovtype ((o)))
{
case LosuTypeDefine_string:
{
ovIstr (o)->marked = 1;
break;
}
case LosuTypeDefine_function:
{
__losuGcmarkFunc (gc, ovfunc (o));
break;
}
case LosuTypeDefine_unit:
{
if (!(ovhash (o)->mark != ovhash (o)))
{
ovhash (o)->mark = gc->unit;
gc->unit = ovhash (o);
}
break;
}
case LosuTypeDefine_coroutine:
{
ovcoro (o)->marked = 1;
break;
}
case LosuTypeDefine_callinfo:
```

```
{
if (ovcall (o)->isMalloc)
ovcall (o)->marked = 1;
__losuGcmarkFunc (gc, ovcall (o)->func);
break;
}
default:
break;
}
}
static void
__losuGcmarkStack (LosuVm *vm, _inlineGc *gc)
{
LosuObj *o;
if (vm->stack == vm->mstack)
vm->mtop = vm->top;
for (o = vm->mstack; o < vm->mtop; o++)
__losuGcmarkObj (gc, o);
_inlineCoroutine *coro;
for (coro = vm->coropool; coro; coro = coro->next)
if (coro->sta != LosuCoroStateEND)
for (o = coro->stack; o < coro->top; o++)
__losuGcmarkObj (gc, o);
}
static void
__losuGcmarkFunc (_inlineGc *gc, _inlineFunction *f)
{
if (f->mark == f)
{
if (!f->isC)
__losuGcmarkScode (f->func.sdef);
f->mark = gc->func;
gc->func = f;
}
}
static void
__losuGcmarkScode (_inlineScode *f)
{
if (!f->marked)
{
f->marked = 1;
f->src->marked = 1;
for (int32_t i = 0; i < f->nlcstr; i++)
f->lcstr[i]->marked = 1;
for (int32_t i = 0; i < f->nlcscode; i++)
__losuGcmarkScode (f->lcscode[i]);
for (int32_t i = 0; i < f->nlocalvar; i++)
f->localvar[i].name->marked = 1;
}
}
```

```
#endif
#ifndef DEFINE_SOURCE_LOSU_GC_C_GCCOLLECT
#define DEFINE_SOURCE_LOSU_GC_C_GCCOLLECT
static void
__losuGcCollect (LosuVm *vm)
{
__losuGcMark (vm);
__losu_gc_collect (vm, 0);
vm->gcDymax *= 2;
}
static void
__losuGccollectScode (LosuVm *vm)
{
_inlineScode **f = &vm->inspool;
_inlineScode *next;
while ((next = *f) != NULL)
{
if (next->marked)
{
next->marked = 0;
f = &next->next;
}
else
{
*f = next->next;
__losu_objFunc_scodeFree (vm, next);
}
}
}
static void
__losuGccollectFunc (LosuVm *vm)
{
_inlineFunction **f = &vm->funcpool;
_inlineFunction *next;
while ((next = *f) != NULL)
{
if (next->mark != next)
{
next->mark = next;
f = &next->next;
}
else
{
*f = next->next;
__losu_objFunc_free (vm, next);
}
}
}
static void
__losuGccollectUnit (LosuVm *vm)
```

```
{
_inlineHash **h = &vm->hashpool;
_inlineHash *next;
while ((next = *h) != NULL)
{
if (next->mark != next)
{
next->mark = next;
h = &next->next;
}
else
{
*h = next->next;
__losu_objUnit_free (vm, next);
}
}
}
static void
__losuGccollectString (LosuVm *vm, _l_bool all)
{
#define sizestring(l) ((sizeof (_inlineString) + (l + 1 - 4) * sizeof (char)))
for (int32_t i = 0; i < vm->strpool.size; i++)
{
_inlineString **s = &vm->strpool.strobj[i];
_inlineString *next;
while ((next = *s) != NULL)
{
if (next->marked && !all)
{
if (next->marked < CONSTMARK)
next->marked = 0;
s = &next->next;
}
else
{
*s = next->next;
vm->strpool.nsize--;
vm->nblocks -= sizestring (next->len);
__losu_mem_free (vm, next);
}
}
}
if (vm->strpool.nsize < vm->strpool.size / 4 && vm->strpool.size > 8)
__losu_objStringPool_resize (vm, &vm->strpool, vm->strpool.size / 2);
#undef sizestring
}
static void
__losuGccollectCallinfo (LosuVm *vm)
{
_inlineCallinfo *cobj = vm->callpool;
```

```
while (cobj != NULL)
{
if (cobj->marked == 1)
{
cobj->marked = 0;
cobj = cobj->next;
}
else
{
_inlineCallinfo *tmp = cobj;
cobj = cobj->next;
__losu_objCallinfo_free (vm, tmp);
}
}
}
static void
__losuGccollectCoroutine (LosuVm *vm)
{
_inlineCoroutine *cobj = vm->coropool;
while (cobj != NULL)
{
if (cobj->marked == 1)
{
cobj->marked = 0;
cobj = cobj->next;
}
else
{
_inlineCoroutine *tmp = cobj;
cobj = cobj->next;
__losu_objCoroutine_free (vm, tmp);
}
}
}
static void
__losuGcAdjbuff (LosuVm *vm)
{
if (vm->bufftmp)
{
vm->nblocks -= vm->nbufftmp * sizeof (char);
vm->nbufftmp = 0;
__losu_mem_free (vm, vm->bufftmp);
vm->bufftmp = NULL;
}
}
#endif
#endif
#ifndef DEFINE_SOURCE_LOSU_MALLOC_C
#define DEFINE_SOURCE_LOSU_MALLOC_C
#include "losu_malloc.h"
```

```
#include "losu.h"
#include "losu_errmsg.h"
#include <stdio.h>
#include <stdlib.h>
void *
__losu_mem_realloc (LosuVm *vm, void *old, _l_size_t size)
{
void *new = NULL;
if (size == 0)
{
if (old != NULL)
__config_losucore_mem_free (old);
return NULL;
}
new = (void *)(__config_losucore_mem_realloc (old, size));
if (new == NULL)
{
if (vm)
vm_error (vm, __config_losucore_errmsg_msgReallocFailed, old, size);
return NULL;
}
return new;
}
void *
__losu_mem_grow (LosuVm *vm, void *block, _l_size_t i_num, int inc,
_l_size_t size, _l_size_t limit, const char *errormsg)
{
_l_size_t newn = i_num + inc;
if (newn >= limit)
vm_error (vm, errormsg);
return __losu_mem_realloc (vm, block, newn * size);
}
#endif
#ifndef DEFINE_SOURCE_LOSU_OBJECT_C
#define DEFINE_SOURCE_LOSU_OBJECT_C
#include "losu_object.h"
#include "losu.h"
#include "losu_errmsg.h"
#include "losu_malloc.h"
#include <ctype.h>
#include <stdarg.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifndef DEFINE_SOURCE_LOSU_OBJECT_C_OBJECT
#define DEFINE_SOURCE_LOSU_OBJECT_C_OBJECT
const LosuObj _inlineNullObj = {
.type = LosuTypeDefine_null,
.value = {NULL},
```

```
};
_l_bool
__losu_object_isObjEqual(const LosuObj *t1, const LosuObj *t2)
{
if (ovtype(t1) != ovtype(t2))
return 0;
switch (ovtype(t1))
{
case LosuTypeDefine_string:
{
if (ovIstr(t1)->hash != ovIstr(t2)->hash)
return 0;
if (ovIstr(t1)->len != ovIstr(t2)->len)
return 0;
return !memcmp(ovSstr(t1), ovSstr(t2), ovIstr(t1)->len);
break;
}
case LosuTypeDefine_number:
{
return ovnumber(t1) == ovnumber(t2);
break;
}
case LosuTypeDefine_unit:
{
return ovhash(t1) == ovhash(t2);
break;
}
case LosuTypeDefine_function:
{
return ovfunc(t1) == ovfunc(t2);
break;
}
case LosuTypeDefine_coroutine:
{
return ovcoro(t1) == ovcoro(t2);
break;
}
default:
return 0;
}
}
_l_bool
__losu_object_isObjLess(const LosuObj *t1, const LosuObj *t2)
{
if (ovtype(t1) != ovtype(t2))
return 0;
switch (ovtype(t1))
{
case LosuTypeDefine_number:
return ovnumber(t1) < ovnumber(t2);
```

```
case LosuTypeDefine_string:
{
const void *l = (const void *)(ovSstr(t1));
const void *r = (const void *)(ovSstr(t2));
_l_size_t ll = ovIstr(t1)->len;
_l_size_t lr = ovIstr(t2)->len;
_l_size_t ml = ll < lr ? ll : lr;
int32_t i = memcmp(l, r, ml);
return i < 0 ? 1 : ((i == 0) ? (ll < lr) : 0);
}
default:
return 0;
break;
}
return 0;
}
_l_bool
__losu_object_str2num(const char *s, _l_number *n)
{
char *endp;
_l_number res = ostr2num(s, &endp);
if (endp == s)
return 0;
while (isspace((uint8_t)*endp))
endp++;
if (*endp != '\0')
return 0;
*n = res;
return 1;
}
#endif
#ifndef DEFINE_SOURCE_LOSU_OBJECT_C_STRING
#define DEFINE_SOURCE_LOSU_OBJECT_C_STRING
#define sizestring(l) ((sizeof(_inlineString) + (l + 1 - 4) * sizeof(char)))
static _l_hash losuhash(const char *s, _l_size_t l);
static _l_hash
losuhash(const char *s, _l_size_t len)
{
_l_size_t h = len;
_l_size_t step = (len >> 5) | 1;
for (; len >= step; len -= step)
h = h ^ ((h << 5) + (h >> 2) + (unsigned char)*(s++));
return (_l_hash)h;
}
void __losu_objStringPool_init(LosuVm *vm)
{
vm->strpool.strobj = __losu_mem_newvector(vm, 1, _inlineString *);
vm->nblocks += sizeof(_inlineString *);
vm->strpool.size = 1;
vm->strpool.nsize = 0;
```

```
vm->strpool.strobj[0] = NULL;
}
void __losu_objStringPool_resize(LosuVm *vm, __losuvmStrseg *strpool, _l_hash
s)
{
_inlineString **newstrobj = __losu_mem_newvector(vm, s, _inlineString *);
memset(newstrobj, 0, s * sizeof(_inlineString *));
for (_l_hash i = 0; i < strpool->size; i++)
{
_inlineString *p = strpool->strobj[i];
while (p)
{
_inlineString *next = p->next;
_l_hash h = p->hash;
_l_hash h1 = h & (s - 1);
p->next = newstrobj[h1];
newstrobj[h1] = p;
p = next;
}
}
__losu_mem_free(vm, strpool->strobj);
vm->nblocks += (s - strpool->size) * sizeof(_inlineString *);
strpool->size = s;
strpool->strobj = newstrobj;
}
void __losu_objStringPool_deinit(LosuVm *vm)
{
vm->nblocks -= vm->strpool.size * sizeof(_inlineString *);
__losu_mem_free(vm, vm->strpool.strobj);
}
_inlineString *
__losu_objString_new(LosuVm *vm, const char *s)
{
return __losu_objString_newstr(vm, s, strlen(s));
}
_inlineString *
__losu_objString_newconst(LosuVm *vm, const char *s)
{
_inlineString *ts = __losu_objString_newstr(vm, s, strlen(s));
if (!ts->marked)
ts->marked = CONSTMARK;
return ts;
}
_inlineString *
__losu_objString_newstr(LosuVm *vm, const char *s, _l_size_t len)
{
_l_hash h = losuhash(s, len);
_l_hash h1 = h & (vm->strpool.size - 1);
_inlineString *ts;
for (ts = vm->strpool.strobj[h1]; ts; ts = ts->next)
```

```
{
if (ts->hash == h)
if (ts->len == len && (memcmp(s, ts->str, len) == 0))
return ts;
}
ts = (_inlineString *)__losu_mem_malloc(vm, sizestring(len));
ts->marked = 0;
ts->next = NULL;
ts->len = len;
ts->hash = h;
ts->cstidx = 0;
memcpy(ts->str, s, len);
ts->str[len] = '\0';
vm->nblocks += sizestring(len);
ts->next = vm->strpool.strobj[h1];
vm->strpool.strobj[h1] = ts;
vm->strpool.nsize++;
if (vm->strpool.nsize > vm->strpool.size && vm->strpool.size <
__config_losucore_vm_strpoolMax / 2 + 1)
__losu_objStringPool_resize(vm, &vm->strpool, vm->strpool.size * 2);
return ts;
}
#undef sizestring
#endif
#ifndef DEFINE_SOURCE_LOSU_OBJECT_C_FUNCTION
#define DEFINE_SOURCE_LOSU_OBJECT_C_FUNCTION
#define sizeInlineFunction(n) \
(sizeof(_inlineFunction) + sizeof(LosuObj) * ((n)))
#define sizeInlineScode(f) \
((sizeof(_inlineScode) + f->nlcnum * sizeof(_l_number) + f->nlcstr *
sizeof(_inlineString *) + f->nlcscode * sizeof(_inlineScode *) + f->ncode *
sizeof(vmInstruction) + f->nlocalvar * sizeof(_inlineLocalvar) + f->nlineinfo
* sizeof(int32_t)))
_inlineScode *
__losu_objFunc_scodeNew(LosuVm *vm)
{
_inlineScode *f = __losu_mem_new(vm, _inlineScode);
memset((void *)f, 0, sizeof(_inlineScode));
f->marked = 0;
f->next = vm->inspool;
vm->inspool = f;
return f;
}
void __losu_objFunc_scodeFree(LosuVm *vm, _inlineScode *f)
{
if (f->ncode > 0)
vm->nblocks -= sizeInlineScode(f);
__losu_mem_free(vm, f->code);
__losu_mem_free(vm, f->localvar);
__losu_mem_free(vm, f->lcstr);
```

```
__losu_mem_free(vm, f->lcnum);
__losu_mem_free(vm, f->lcscode);
__losu_mem_free(vm, f->lineinfo);
__losu_mem_free(vm, f);
}
_inlineFunction *
__losu_objFunc_new(LosuVm *vm, int32_t issnum)
{
_l_size_t size = sizeInlineFunction(issnum);
_inlineFunction *f = (_inlineFunction *)__losu_mem_malloc(vm, size);
f->next = vm->funcpool;
vm->funcpool = f;
f->mark = f;
f->nclosure = issnum;
vm->nblocks += size;
return f;
}
void __losu_objFunc_free(LosuVm *vm, _inlineFunction *f)
{
vm->nblocks -= sizeInlineFunction(f->nclosure);
__losu_mem_free(vm, f);
}
#undef sizeInlineFunction
#undef sizeInlineScode
#endif
#ifndef DEFINE_SOURCE_LOSU_OBJECT_C_UNIT
#define DEFINE_SOURCE_LOSU_OBJECT_C_UNIT
static LosuNode *__losuObjectUnitMainpoint(_inlineHash *t, LosuObj *key);
static void __losuObjectUnitSetvector(LosuVm *vm, _inlineHash *t,
int32_t size);
static int32_t __losuObjectUnitGetnsize(_inlineHash *t);
static void __losuObjectUnitRehash(LosuVm *vm, _inlineHash *t);
#define sizeInlineHash(n) (sizeof(_inlineHash) + sizeof(LosuNode) * (n))
const LosuObj *
__losu_objUnit_getglobal(LosuVm *vm, const char *name)
{
return __losu_objUnit_getstr(vm->global, __losu_objString_new(vm, name));
}
const LosuObj *
__losu_objUnit_get(_inlineHash *t, LosuObj *key)
{
switch (ovtype(key))
{
case LosuTypeDefine_number:
return __losu_objUnit_getnum(t, ovnumber(key));
case LosuTypeDefine_string:
return __losu_objUnit_getstr(t, ovIstr(key));
default:
return __losu_objUnit_getany(t, key);
}
```

```
}
const LosuObj *
__losu_objUnit_getany(_inlineHash *t, LosuObj *key)
{
LosuNode *n = __losuObjectUnitMainpoint(t, key);
if (n)
{
do
{
if (__losu_object_isObjEqual((const LosuObj *)(key),
(const LosuObj *)(&n->key)))
return (const LosuObj *)&n->value;
n = n->next;
} while (n);
}
return (const LosuObj *)&_inlineNullObj;
}
const LosuObj *
__losu_objUnit_getnum(_inlineHash *t, _l_number key)
{
LosuNode *n = &t->node[(_l_hash)(key) & (t->size - 1)];
do
{
if (ovtype((&n->key)) == LosuTypeDefine_number && ovnumber((&n->key)) == key)
return (const LosuObj *)&n->value;
n = n->next;
} while (n);
return (const LosuObj *)&_inlineNullObj;
}
const LosuObj *
__losu_objUnit_getstr(_inlineHash *t, _inlineString *key)
{
LosuNode *n = &t->node[key->hash & (t->size - 1)];
do
{
if (ovtype((&n->key)) == LosuTypeDefine_string && ovIstr((&n->key))->hash ==
key->hash)
if (!strcmp(ovIstr((&n->key))->str, key->str))
return (const LosuObj *)&n->value;
n = n->next;
} while (n);
return (const LosuObj *)&_inlineNullObj;
}
const LosuNode *
__losu_objUnit_getnext(_inlineHash *t, LosuObj *key)
{
int32_t i;
if (ovtype(key) == LosuTypeDefine_null)
i = 0;
else
```

```
{
const LosuObj *v = __losu_objUnit_get(t, key);
if (v == &_inlineNullObj)
return NULL;
i = (int32_t)(((char *)v - (char *)(&t->node[0].value)) / sizeof(LosuNode) + 1);
}
for (; i < t->size; i++)
{
LosuNode *n = &(t->node[i]);
if (ovtype((&n->value)) != LosuTypeDefine_null)
return n;
}
return NULL;
}
_inlineHash *
__losu_objUnit_new(LosuVm *vm, int32_t size)
{
_inlineHash *t = __losu_mem_new(vm, _inlineHash);
t->next = vm->hashpool;
vm->hashpool = t;
t->mark = t;
t->size = 0;
vm->nblocks += sizeInlineHash(0);
t->node = NULL;
{
int32_t p = 4;
while (p <= size)
p <<= 1;
size = p;
}
__losuObjectUnitSetvector(vm, t, size);
return t;
}
void __losu_objUnit_remove(_inlineHash *t, LosuObj *key)
{
ovtype(key) = LosuTypeDefine_number;
ovnumber(key) = 0;
}
void __losu_objUnit_free(LosuVm *vm, _inlineHash *t)
{
vm->nblocks -= sizeInlineHash(t->size);
__losu_mem_free(vm, t->node);
__losu_mem_free(vm, t);
}
LosuObj *
__losu_objUnit_set(LosuVm *vm, _inlineHash *t, LosuObj *key)
{
LosuNode *mp = __losuObjectUnitMainpoint(t, key);
LosuNode *n = mp;
if (!n)
```

```
vm_error(vm, __config_losucore_errmsg_msgInvalidUnitIndex);
do
{
if (__losu_object_isObjEqual(key, &n->key))
return &n->value;
n = n->next;
} while (n);
if (ovtype((&mp->key)) != LosuTypeDefine_null)
{
LosuNode *oth;
n = t->free;
if (mp > n && (oth = __losuObjectUnitMainpoint(t, &mp->key)) != mp)
{
while (oth->next != mp)
oth = oth->next;
oth->next = n;
*n = *mp;
mp->next = NULL;
}
else
{
n->next = mp->next;
mp->next = n;
mp = n;
}
}
mp->key = *key;
while (1)
{
if (ovtype((&t->free->key)) == LosuTypeDefine_null)
return &mp->value;
else if (t->free == t->node)
break;
else
(t->free)--;
}
__losuObjectUnitRehash(vm, t);
return __losu_objUnit_set(vm, t, key);
}
LosuObj *
__losu_objUnit_setnum(LosuVm *vm, _inlineHash *t, _l_number key)
{
LosuObj idx = (LosuObj){
.type = LosuTypeDefine_number,
.value.num = key,
};
return __losu_objUnit_set(vm, t, &idx);
}
LosuObj *
__losu_objUnit_setstr(LosuVm *vm, _inlineHash *t, _inlineString *key)
```

```
{
LosuObj idx = (LosuObj){
.type = LosuTypeDefine_string,
.value.str = key,
};
return __losu_objUnit_set(vm, t, &idx);
}
static LosuNode *
__losuObjectUnitMainpoint(_inlineHash *t, LosuObj *key)
{
_l_size_t h;
switch (key->type)
{
case LosuTypeDefine_number:
h = (_l_size_t)(ovnumber(key));
break;
case LosuTypeDefine_string:
h = (_l_size_t)(ovIstr(key)->hash);
break;
case LosuTypeDefine_unit:
h = (_l_size_t)((size_t)(ovhash(key)));
break;
case LosuTypeDefine_function:
h = (_l_size_t)((size_t)(ovfunc(key)));
break;
default:
return NULL;
}
return &t->node[h & (t->size - 1)];
}
static void
__losuObjectUnitSetvector(LosuVm *vm, _inlineHash *t, int32_t size)
{
int32_t i;
if (size > INT32_MAX)
vm_error(vm, __config_losucore_errmsg_msgVectorOverflow);
t->node = __losu_mem_newvector(vm, size, LosuNode);
for (i = 0; i < size; i++)
{
ovtype((&t->node[i].key)) = ovtype((&t->node[i].value)) = LosuTypeDefine_null;
t->node[i].next = NULL;
}
vm->nblocks += (sizeInlineHash(size)) - (sizeInlineHash(t->size));
t->size = size;
t->free = &t->node[size - 1];
}
static int32_t
__losuObjectUnitGetnsize(_inlineHash *t)
{
LosuNode *v = t->node;
```

```
int32_t s = t->size;
int32_t use = 0;
int32_t i;
for (i = 0; i < s; i++)
if (ovtype((&v[i].value)) != LosuTypeDefine_null)
use++;
return use;
}
static void
__losuObjectUnitRehash(LosuVm *vm, _inlineHash *t)
{
int32_t osize = t->size;
LosuNode *onode = t->node;
int32_t uszie = __losuObjectUnitGetnsize(t);
int32_t i;
if (uszie >= osize - osize / 4)
__losuObjectUnitSetvector(vm, t, osize * 2);
else if (uszie <= osize / 4 && osize > 4)
__losuObjectUnitSetvector(vm, t, osize / 2);
else
__losuObjectUnitSetvector(vm, t, osize);
for (i = 0; i < osize; i++)
if (ovtype((&((onode + i)->value))) != LosuTypeDefine_null)
*__losu_objUnit_set(vm, t, &(onode + i)->key) = ((LosuNode *)(onode +
i))->value;
__losu_mem_free(vm, onode);
}
#undef sizeInlineHash
#endif
#ifndef DEFINE_SOURCE_LOSU_OBJECT_C_CALLINFO
#define DEFINE_SOURCE_LOSU_OBJECT_C_CALLINFO
_inlineCallinfo *
__losu_objCallinfo_new(LosuVm *vm)
{
_inlineCallinfo *nc = __losu_mem_new(vm, _inlineCallinfo);
vm->nblocks += sizeof(_inlineCallinfo);
memset(nc, 0, sizeof(_inlineCallinfo));
nc->pre = NULL;
nc->next = vm->callpool;
if (vm->callpool)
vm->callpool->pre = nc;
vm->callpool = nc;
nc->isMalloc = 1;
return nc;
}
void __losu_objCallinfo_free(LosuVm *vm, _inlineCallinfo *c)
{
if (!c->isMalloc)
return;
if (c->pre)
```

```
c->pre->next = c->next;
if (c->next)
c->next->pre = c->pre;
if (c == vm->callpool)
vm->callpool = c->next;
__losu_mem_free(vm, (_inlineCallinfo *)c);
vm->nblocks -= sizeof(_inlineCallinfo);
}
#endif
#ifndef DEFINE_SOURCE_LOSU_OBJECT_C_COROUTINE
#define DEFINE_SOURCE_LOSU_OBJECT_C_COROUTINE
_inlineCoroutine *
__losu_objCoroutine_new(LosuVm *vm)
{
_inlineCoroutine *nc = __losu_mem_new(vm, _inlineCoroutine);
vm->nblocks += sizeof(_inlineCoroutine);
memset(nc, 0, sizeof(_inlineCoroutine));
nc->pre = NULL;
nc->next = vm->coropool;
if (vm->coropool)
vm->coropool->pre = nc;
vm->coropool = nc;
return nc;
}
void __losu_objCoroutine_free(LosuVm *vm, _inlineCoroutine *co)
{
if (co->pre)
co->pre->next = co->next;
if (co->next)
co->next->pre = co->pre;
if (co == vm->coropool)
vm->coropool = co->next;
__losu_mem_free(vm, co);
vm->nblocks -= sizeof(_inlineCoroutine);
}
#endif
#endif
function (global, factory) {
typeof exports === 'object' && typeof module !== 'undefined' ? module.exports
= factory() :
typeof define === 'function' && define.amd ? define(factory) :
(global = global || self, global.CodeMirror = factory());
}(this, (function () { 'use strict';
var userAgent = navigator.userAgent;
var platform = navigator.platform;
var gecko = /gecko\/\d/i.test(userAgent);
var ie_upto10 = /MSIE \d/.test(userAgent);
var ie_11up = /Trident\/(?:[7-9]|\d{2,})\..*rv:(\d+)/.exec(userAgent);
var edge = /Edge\/(\d+)/.exec(userAgent);
var ie = ie_upto10 || ie_11up || edge;
```

```
var ie_version = ie && (ie_upto10 ? document.documentMode || 6 : +(edge ||
ie_11up)[1]);
var webkit = !edge && /WebKit\
var qtwebkit = webkit && /Qt\/\d+\.\d+/.test(userAgent);
var chrome = !edge && /Chrome\/(\d+)/.exec(userAgent);
var chrome_version = chrome && +chrome[1];
var presto = /Opera\
var safari = /Apple Computer/.test(navigator.vendor);
var mac_geMountainLion = /Mac OS X 1\d\D([8-9]|\d\d)\D/.test(userAgent);
var phantom = /PhantomJS/.test(userAgent);
var ios = safari && (/Mobile\/\w+/.test(userAgent) || navigator.maxTouchPoints >
2);
var android = /Android/.test(userAgent);
var mobile = ios || android || /webOS|BlackBerry|Opera Mini|Opera
Mobi|IEMobile/i.test(userAgent);
var mac = ios || /Mac/.test(platform);
var chromeOS = /\bCrOS\b/.test(userAgent);
var windows = /win/i.test(platform);
var presto_version = presto && userAgent.match(/Version\/(\d*\.\d*)/);
if (presto_version) { presto_version = Number(presto_version[1]); }
if (presto_version && presto_version >= 15) { presto = false; webkit = true; }
var flipCtrlCmd = mac && (qtwebkit || presto && (presto_version == null ||
presto_version < 12.11));
var captureRightClick = gecko || (ie && ie_version >= 9);
function classTest(cls) { return new RegExp("(^|\\s)" + cls + "(?:$|\\s)\\s*") }
var rmClass = function(node, cls) {
var current = node.className;
var match = classTest(cls).exec(current);
if (match) {
var after = current.slice(match.index + match[0].length);
node.className = current.slice(0, match.index) + (after ? match[1] + after : "");
}
};
function removeChildren(e) {
for (var count = e.childNodes.length; count > 0; --count)
{ e.removeChild(e.firstChild); }
return e
}
function removeChildrenAndAdd(parent, e) {
return removeChildren(parent).appendChild(e)
}
function elt(tag, content, className, style) {
var e = document.createElement(tag);
if (className) { e.className = className; }
if (style) { e.style.cssText = style; }
if (typeof content == "string")
{ e.appendChild(document.createTextNode(content)); }
else if (content) { for (var i = 0; i < content.length; ++i)
{ e.appendChild(content[i]); } }
return e
```

```
}
function eltP(tag, content, className, style) {
var e = elt(tag, content, className, style);
e.setAttribute("role", "presentation");
return e
}
var range;
if (document.createRange) { range = function(node, start, end, endNode) {
var r = document.createRange();
r.setEnd(endNode || node, end);
r.setStart(node, start);
return r
}; }
else { range = function(node, start, end) {
var r = document.body.createTextRange();
try { r.moveToElementText(node.parentNode); }
catch(e) { return r }
r.collapse(true);
r.moveEnd("character", end);
r.moveStart("character", start);
return r
}; }
function contains(parent, child) {
if (child.nodeType == 3)
{ child = child.parentNode; }
if (parent.contains)
{ return parent.contains(child) }
do {
if (child.nodeType == 11) { child = child.host; }
if (child == parent) { return true }
} while (child = child.parentNode)
}
function activeElt(doc) {
var activeElement;
try {
activeElement = doc.activeElement;
} catch(e) {
activeElement = doc.body || null;
}
while (activeElement && activeElement.shadowRoot &&
activeElement.shadowRoot.activeElement)
{ activeElement = activeElement.shadowRoot.activeElement; }
return activeElement
}
function addClass(node, cls) {
var current = node.className;
if (!classTest(cls).test(current)) { node.className += (current ? " " : "") +
cls; }
}
function joinClasses(a, b) {
```

```
var as = a.split(" ");
for (var i = 0; i < as.length; i++)
{ if (as[i] && !classTest(as[i]).test(b)) { b += " " + as[i]; } }
return b
}
var selectInput = function(node) { node.select(); };
if (ios)
{ selectInput = function(node) { node.selectionStart = 0; node.selectionEnd =
node.value.length; }; }
else if (ie)
{ selectInput = function(node) { try { node.select(); } catch(_e) {} }; }
function doc(cm) { return cm.display.wrapper.ownerDocument }
function win(cm) { return doc(cm).defaultView }
function bind(f) {
var args = Array.prototype.slice.call(arguments, 1);
return function(){return f.apply(null, args)}
}
function copyObj(obj, target, overwrite) {
if (!target) { target = {}; }
for (var prop in obj)
{ if (obj.hasOwnProperty(prop) && (overwrite !== false
|| !target.hasOwnProperty(prop)))
{ target[prop] = obj[prop]; } }
return target
}
function countColumn(string, end, tabSize, startIndex, startValue) {
if (end == null) {
end = string.search(/[^\s\u00a0]/);
if (end == -1) { end = string.length; }
}
for (var i = startIndex || 0, n = startValue || 0;;) {
var nextTab = string.indexOf("\t", i);
if (nextTab < 0 || nextTab >= end)
{ return n + (end - i) }
n += nextTab - i;
n += tabSize - (n % tabSize);
i = nextTab + 1;
}
}
var Delayed = function() {
this.id = null;
this.f = null;
this.time = 0;
this.handler = bind(this.onTimeout, this);
};
Delayed.prototype.onTimeout = function (self) {
self.id = 0;
if (self.time <= +new Date) {
self.f();
} else {
```

```
setTimeout(self.handler, self.time - +new Date);
}
};
Delayed.prototype.set = function (ms, f) {
this.f = f;
var time = +new Date + ms;
if (!this.id || time < this.time) {
clearTimeout(this.id);
this.id = setTimeout(this.handler, ms);
this.time = time;
}
};
function indexOf(array, elt) {
for (var i = 0; i < array.length; ++i)
{ if (array[i] == elt) { return i } }
return -1
}
var scrollerGap = 50;
var Pass = {toString: function(){return "CodeMirror.Pass"}};
var sel_dontScroll = {scroll: false}, sel_mouse = {origin: "*mouse"}, sel_move
= {origin: "+move"};
function findColumn(string, goal, tabSize) {
for (var pos = 0, col = 0;;) {
var nextTab = string.indexOf("\t", pos);
if (nextTab == -1) { nextTab = string.length; }
var skipped = nextTab - pos;
if (nextTab == string.length || col + skipped >= goal)
{ return pos + Math.min(skipped, goal - col) }
col += nextTab - pos;
col += tabSize - (col % tabSize);
pos = nextTab + 1;
if (col >= goal) { return pos }
}
}
var spaceStrs = [""];
function spaceStr(n) {
while (spaceStrs.length <= n)
{ spaceStrs.push(lst(spaceStrs) + " "); }
return spaceStrs[n]
}
function lst(arr) { return arr[arr.length-1] }
function map(array, f) {
var out = [];
for (var i = 0; i < array.length; i++) { out[i] = f(array[i], i); }
return out
}
function insertSorted(array, value, score) {
var pos = 0, priority = score(value);
while (pos < array.length && score(array[pos]) <= priority) { pos++; }
array.splice(pos, 0, value);
```

```
}
function nothing() {}
function createObj(base, props) {
var inst;
if (Object.create) {
inst = Object.create(base);
} else {
nothing.prototype = base;
inst = new nothing();
}
if (props) { copyObj(props, inst); }
return inst
}
var nonASCIISingleCaseWordChar =
/[\u00df\u0587\u0590-\u05f4\u0600-\u06ff\u3040-\u309f\u30a0-\u30ff\u3400-\u4
db5\u4e00-\u9fcc\uac00-\ud7af]/;
function isWordCharBasic(ch) {
return /\w/.test(ch) || ch > "\x80" &&
(ch.toUpperCase() != ch.toLowerCase() || nonASCIISingleCaseWordChar.test(ch))
}
function isWordChar(ch, helper) {
if (!helper) { return isWordCharBasic(ch) }
if (helper.source.indexOf("\\w") > -1 && isWordCharBasic(ch)) { return true }
return helper.test(ch)
}
function isEmpty(obj) {
for (var n in obj) { if (obj.hasOwnProperty(n) && obj[n]) { return false } }
return true
}
var extendingChars =
/[\u0300-\u036f\u0483-\u0489\u0591-\u05bd\u05bf\u05c1\u05c2\u05c4\u05c5\u05c
7\u0610-\u061a\u064b-\u065e\u0670\u06d6-\u06dc\u06de-\u06e4\u06e7\u06e8\u06e
a-\u06ed\u0711\u0730-\u074a\u07a6-\u07b0\u07eb-\u07f3\u0816-\u0819\u081b-\u0
823\u0825-\u0827\u0829-\u082d\u0900-\u0902\u093c\u0941-\u0948\u094d\u0951-\u
0955\u0962\u0963\u0981\u09bc\u09be\u09c1-\u09c4\u09cd\u09d7\u09e2\u09e3\u0a0
1\u0a02\u0a3c\u0a41\u0a42\u0a47\u0a48\u0a4b-\u0a4d\u0a51\u0a70\u0a71\u0a75\u
0a81\u0a82\u0abc\u0ac1-\u0ac5\u0ac7\u0ac8\u0acd\u0ae2\u0ae3\u0b01\u0b3c\u0b3
e\u0b3f\u0b41-\u0b44\u0b4d\u0b56\u0b57\u0b62\u0b63\u0b82\u0bbe\u0bc0\u0bcd\u
0bd7\u0c3e-\u0c40\u0c46-\u0c48\u0c4a-\u0c4d\u0c55\u0c56\u0c62\u0c63\u0cbc\u0
cbf\u0cc2\u0cc6\u0ccc\u0ccd\u0cd5\u0cd6\u0ce2\u0ce3\u0d3e\u0d41-\u0d44\u0d4d
\u0d57\u0d62\u0d63\u0dca\u0dcf\u0dd2-\u0dd4\u0dd6\u0ddf\u0e31\u0e34-\u0e3a\u
0e47-\u0e4e\u0eb1\u0eb4-\u0eb9\u0ebb\u0ebc\u0ec8-\u0ecd\u0f18\u0f19\u0f35\u0
f37\u0f39\u0f71-\u0f7e\u0f80-\u0f84\u0f86\u0f87\u0f90-\u0f97\u0f99-\u0fbc\u0
fc6\u102d-\u1030\u1032-\u1037\u1039\u103a\u103d\u103e\u1058\u1059\u105e-\u10
60\u1071-\u1074\u1082\u1085\u1086\u108d\u109d\u135f\u1712-\u1714\u1732-\u173
4\u1752\u1753\u1772\u1773\u17b7-\u17bd\u17c6\u17c9-\u17d3\u17dd\u180b-\u180d
\u18a9\u1920-\u1922\u1927\u1928\u1932\u1939-\u193b\u1a17\u1a18\u1a56\u1a58-\
u1a5e\u1a60\u1a62\u1a65-\u1a6c\u1a73-\u1a7c\u1a7f\u1b00-\u1b03\u1b34\u1b36-\
u1b3a\u1b3c\u1b42\u1b6b-\u1b73\u1b80\u1b81\u1ba2-\u1ba5\u1ba8\u1ba9\u1c2c-\u
1c33\u1c36\u1c37\u1cd0-\u1cd2\u1cd4-\u1ce0\u1ce2-\u1ce8\u1ced\u1dc0-\u1de6\u
```

```
1dfd-\u1dff\u200c\u200d\u20d0-\u20f0\u2cef-\u2cf1\u2de0-\u2dff\u302a-\u302f\
u3099\u309a\ua66f-\ua672\ua67c\ua67d\ua6f0\ua6f1\ua802\ua806\ua80b\ua825\ua8
26\ua8c4\ua8e0-\ua8f1\ua926-\ua92d\ua947-\ua951\ua980-\ua982\ua9b3\ua9b6-\ua
9b9\ua9bc\uaa29-\uaa2e\uaa31\uaa32\uaa35\uaa36\uaa43\uaa4c\uaab0\uaab2-\uaab
4\uaab7\uaab8\uaabe\uaabf\uaac1\uabe5\uabe8\uabed\udc00-\udfff\ufb1e\ufe00-\
ufe0f\ufe20-\ufe26\uff9e\uff9f]/;
function isExtendingChar(ch) { return ch.charCodeAt(0) >= 768 &&
extendingChars.test(ch) }
function skipExtendingChars(str, pos, dir) {
while ((dir < 0 ? pos > 0 : pos < str.length) && isExtendingChar(str.charAt(pos)))
{ pos += dir; }
return pos
}
function findFirst(pred, from, to) {
var dir = from > to ? -1 : 1;
for (;;) {
if (from == to) { return from }
var midF = (from + to) / 2, mid = dir < 0 ? Math.ceil(midF) : Math.floor(midF);
if (mid == from) { return pred(mid) ? from : to }
if (pred(mid)) { to = mid; }
else { from = mid + dir; }
}
}
function iterateBidiSections(order, from, to, f) {
if (!order) { return f(from, to, "ltr", 0) }
var found = false;
for (var i = 0; i < order.length; ++i) {
var part = order[i];
if (part.from < to && part.to > from || from == to && part.to == from) {
f(Math.max(part.from, from), Math.min(part.to, to), part.level == 1 ? "rtl" :
"ltr", i);
found = true;
}
}
if (!found) { f(from, to, "ltr"); }
}
var bidiOther = null;
function getBidiPartAt(order, ch, sticky) {
var found;
bidiOther = null;
for (var i = 0; i < order.length; ++i) {
var cur = order[i];
if (cur.from < ch && cur.to > ch) { return i }
if (cur.to == ch) {
if (cur.from != cur.to && sticky == "before") { found = i; }
else { bidiOther = i; }
}
if (cur.from == ch) {
if (cur.from != cur.to && sticky != "before") { found = i; }
else { bidiOther = i; }
```

```
}
}
return found != null ? found : bidiOther
}
var bidiOrdering = (function() {
var lowTypes =
"bbbbbbbbbtstwsbbbbbbbbbbbbbbssstwNN%%%NNNNNN,N,N1111111111NNNNNNNLLLLLLLLLL
LLLLLLLLLLLLLLLLLNNNNNNLLLLLLLLLLLLLLLLLLLLLLLLLLNNNNbbbbbbsbbbbbbbbbbbbbbbbb
bbbbbbbbb,N%%%%NNNNLNNNNN%%11NLNNN1LNNNNNLLLLLLLLLLLLLLLLLLLLLLLNLLLLLLLLLLL
LLLLLLLLLLLLLLLLLLLLN";
var arabicTypes =
"nnnnnnNNr%%r,rNNmmmmmmmmmmmmrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrr
mmmmmmmmmmmmmmmmmmmmmmmmmmnnnnnnnnnn%nnrrrmrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrr
rrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrrmmmmmmmmNmmmm
mmrrmmNmmmmrr1111111111";
function charType(code) {
if (code <= 0xf7) { return lowTypes.charAt(code) }
else if (0x590 <= code && code <= 0x5f4) { return "R" }
else if (0x600 <= code && code <= 0x6f9) { return arabicTypes.charAt(code -
0x600) }
else if (0x6ee <= code && code <= 0x8ac) { return "r" }
else if (0x2000 <= code && code <= 0x200b) { return "w" }
else if (code == 0x200c) { return "b" }
else { return "L" }
}
var bidiRE = /[\u0590-\u05f4\u0600-\u06ff\u0700-\u08ac]/;
var isNeutral = /[stwN]/, isStrong = /[LRr]/, countsAsLeft = /[Lb1n]/,
countsAsNum = /[1n]/;
function BidiSpan(level, from, to) {
this.level = level;
this.from = from; this.to = to;
}
return function(str, direction) {
var outerType = direction == "ltr" ? "L" : "R";
if (str.length == 0 || direction == "ltr" && !bidiRE.test(str)) { return false }
var len = str.length, types = [];
for (var i = 0; i < len; ++i)
{ types.push(charType(str.charCodeAt(i))); }
for (var i$1 = 0, prev = outerType; i$1 < len; ++i$1) {
var type = types[i$1];
if (type == "m") { types[i$1] = prev; }
else { prev = type; }
}
for (var i$2 = 0, cur = outerType; i$2 < len; ++i$2) {
var type$1 = types[i$2];
if (type$1 == "1" && cur == "r") { types[i$2] = "n"; }
else if (isStrong.test(type$1)) { cur = type$1; if (type$1 == "r") { types[i$2]
= "R"; } }
}
for (var i$3 = 1, prev$1 = types[0]; i$3 < len - 1; ++i$3) {
```

```
var type$2 = types[i$3];
if (type$2 == "+" && prev$1 == "1" && types[i$3+1] == "1") { types[i$3] = "1"; }
else if (type$2 == "," && prev$1 == types[i$3+1] &&
(prev$1 == "1" || prev$1 == "n")) { types[i$3] = prev$1; }
prev$1 = type$2;
}
for (var i$4 = 0; i$4 < len; ++i$4) {
var type$3 = types[i$4];
if (type$3 == ",") { types[i$4] = "N"; }
else if (type$3 == "%") {
var end = (void 0);
for (end = i$4 + 1; end < len && types[end] == "%"; ++end) {}
var replace = (i$4 && types[i$4-1] == "!") || (end < len && types[end] == "1") ?
"1" : "N";
for (var j = i$4; j < end; ++j) { types[j] = replace; }
i$4 = end - 1;
}
}
for (var i$5 = 0, cur$1 = outerType; i$5 < len; ++i$5) {
var type$4 = types[i$5];
if (cur$1 == "L" && type$4 == "1") { types[i$5] = "L"; }
else if (isStrong.test(type$4)) { cur$1 = type$4; }
}
for (var i$6 = 0; i$6 < len; ++i$6) {
if (isNeutral.test(types[i$6])) {
var end$1 = (void 0);
for (end$1 = i$6 + 1; end$1 < len && isNeutral.test(types[end$1]); ++end$1) {}
var before = (i$6 ? types[i$6-1] : outerType) == "L";
var after = (end$1 < len ? types[end$1] : outerType) == "L";
var replace$1 = before == after ? (before ? "L" : "R") : outerType;
for (var j$1 = i$6; j$1 < end$1; ++j$1) { types[j$1] = replace$1; }
i$6 = end$1 - 1;
}
}
var order = [], m;
for (var i$7 = 0; i$7 < len;) {
if (countsAsLeft.test(types[i$7])) {
var start = i$7;
for (++i$7; i$7 < len && countsAsLeft.test(types[i$7]); ++i$7) {}
order.push(new BidiSpan(0, start, i$7));
} else {
var pos = i$7, at = order.length, isRTL = direction == "rtl" ? 1 : 0;
for (++i$7; i$7 < len && types[i$7] != "L"; ++i$7) {}
for (var j$2 = pos; j$2 < i$7;) {
if (countsAsNum.test(types[j$2])) {
if (pos < j$2) { order.splice(at, 0, new BidiSpan(1, pos, j$2)); at += isRTL; }
var nstart = j$2;
for (++j$2; j$2 < i$7 && countsAsNum.test(types[j$2]); ++j$2) {}
order.splice(at, 0, new BidiSpan(2, nstart, j$2));
at += isRTL;
```

```
pos = j$2;
} else { ++j$2; }
}
if (pos < i$7) { order.splice(at, 0, new BidiSpan(1, pos, i$7)); }
}
}
if (direction == "ltr") {
if (order[0].level == 1 && (m = str.match(/^\s+/))) {
order[0].from = m[0].length;
order.unshift(new BidiSpan(0, 0, m[0].length));
}
if (lst(order).level == 1 && (m = str.match(/\s+$/))) {
lst(order).to -= m[0].length;
order.push(new BidiSpan(0, len - m[0].length, len));
}
}
return direction == "rtl" ? order.reverse() : order
}
})();
function getOrder(line, direction) {
var order = line.order;
if (order == null) { order = line.order = bidiOrdering(line.text, direction); }
return order
}
var noHandlers = [];
var on = function(emitter, type, f) {
if (emitter.addEventListener) {
emitter.addEventListener(type, f, false);
} else if (emitter.attachEvent) {
emitter.attachEvent("on" + type, f);
} else {
var map = emitter._handlers || (emitter._handlers = {});
map[type] = (map[type] || noHandlers).concat(f);
}
};
function getHandlers(emitter, type) {
return emitter._handlers && emitter._handlers[type] || noHandlers
}
function off(emitter, type, f) {
if (emitter.removeEventListener) {
emitter.removeEventListener(type, f, false);
} else if (emitter.detachEvent) {
emitter.detachEvent("on" + type, f);
} else {
var map = emitter._handlers, arr = map && map[type];
if (arr) {
var index = indexOf(arr, f);
if (index > -1)
{ map[type] = arr.slice(0, index).concat(arr.slice(index + 1)); }
}
```

```
}
}
function signal(emitter, type ) {
var handlers = getHandlers(emitter, type);
if (!handlers.length) { return }
var args = Array.prototype.slice.call(arguments, 2);
for (var i = 0; i < handlers.length; ++i) { handlers[i].apply(null, args); }
}
function signalDOMEvent(cm, e, override) {
if (typeof e == "string")
{ e = {type: e, preventDefault: function() { this.defaultPrevented = true; }}; }
signal(cm, override || e.type, cm, e);
return e_defaultPrevented(e) || e.codemirrorIgnore
}
function signalCursorActivity(cm) {
var arr = cm._handlers && cm._handlers.cursorActivity;
if (!arr) { return }
var set = cm.curOp.cursorActivityHandlers || (cm.curOp.cursorActivityHandlers
= []);
for (var i = 0; i < arr.length; ++i) { if (indexOf(set, arr[i]) == -1)
{ set.push(arr[i]); } }
}
function hasHandler(emitter, type) {
return getHandlers(emitter, type).length > 0
}
function eventMixin(ctor) {
ctor.prototype.on = function(type, f) {on(this, type, f);};
ctor.prototype.off = function(type, f) {off(this, type, f);};
}
function e_preventDefault(e) {
if (e.preventDefault) { e.preventDefault(); }
else { e.returnValue = false; }
}
function e_stopPropagation(e) {
if (e.stopPropagation) { e.stopPropagation(); }
else { e.cancelBubble = true; }
}
function e_defaultPrevented(e) {
return e.defaultPrevented != null ? e.defaultPrevented : e.returnValue == false
}
function e_stop(e) {e_preventDefault(e); e_stopPropagation(e);}
function e_target(e) {return e.target || e.srcElement}
function e_button(e) {
var b = e.which;
if (b == null) {
if (e.button & 1) { b = 1; }
else if (e.button & 2) { b = 3; }
else if (e.button & 4) { b = 2; }
}
if (mac && e.ctrlKey && b == 1) { b = 3; }
```

```
return b
}
var dragAndDrop = function() {
if (ie && ie_version < 9) { return false }
var div = elt('div');
return "draggable" in div || "dragDrop" in div
}();
var zwspSupported;
function zeroWidthElement(measure) {
if (zwspSupported == null) {
var test = elt("span", "\u200b");
removeChildrenAndAdd(measure, elt("span", [test,
document.createTextNode("x")]));
if (measure.firstChild.offsetHeight != 0)
{ zwspSupported = test.offsetWidth <= 1 && test.offsetHeight > 2 && !(ie &&
ie_version < 8); }
}
var node = zwspSupported ? elt("span", "\u200b") :
elt("span", "\u00a0", null, "display: inline-block; width: 1px; margin-right:
-1px");
node.setAttribute("cm-text", "");
return node
}
var badBidiRects;
function hasBadBidiRects(measure) {
if (badBidiRects != null) { return badBidiRects }
var txt = removeChildrenAndAdd(measure, document.createTextNode("A\u062eA"));
var r0 = range(txt, 0, 1).getBoundingClientRect();
var r1 = range(txt, 1, 2).getBoundingClientRect();
removeChildren(measure);
if (!r0 || r0.left == r0.right) { return false }
return badBidiRects = (r1.right - r0.right < 3)
}
var splitLinesAuto = "\n\nb".split(/\n/).length != 3 ? function (string) {
var pos = 0, result = [], l = string.length;
while (pos <= l) {
var nl = string.indexOf("\n", pos);
if (nl == -1) { nl = string.length; }
var line = string.slice(pos, string.charAt(nl - 1) == "\r" ? nl - 1 : nl);
var rt = line.indexOf("\r");
if (rt != -1) {
result.push(line.slice(0, rt));
pos += rt + 1;
} else {
result.push(line);
pos = nl + 1;
}
}
return result
} : function (string) { return string.split(/\r\n?|\n/); };
```

```
var hasSelection = window.getSelection ? function (te) {
try { return te.selectionStart != te.selectionEnd }
catch(e) { return false }
} : function (te) {
var range;
try {range = te.ownerDocument.selection.createRange();}
catch(e) {}
if (!range || range.parentElement() != te) { return false }
return range.compareEndPoints("StartToEnd", range) != 0
};
var hasCopyEvent = (function () {
var e = elt("div");
if ("oncopy" in e) { return true }
e.setAttribute("oncopy", "return;");
return typeof e.oncopy == "function"
})();
var badZoomedRects = null;
function hasBadZoomedRects(measure) {
if (badZoomedRects != null) { return badZoomedRects }
var node = removeChildrenAndAdd(measure, elt("span", "x"));
var normal = node.getBoundingClientRect();
var fromRange = range(node, 0, 1).getBoundingClientRect();
return badZoomedRects = Math.abs(normal.left - fromRange.left) > 1
}
var modes = {}, mimeModes = {};
function defineMode(name, mode) {
if (arguments.length > 2)
{ mode.dependencies = Array.prototype.slice.call(arguments, 2); }
modes[name] = mode;
}
function defineMIME(mime, spec) {
mimeModes[mime] = spec;
}
function resolveMode(spec) {
if (typeof spec == "string" && mimeModes.hasOwnProperty(spec)) {
spec = mimeModes[spec];
} else if (spec && typeof spec.name == "string" &&
mimeModes.hasOwnProperty(spec.name)) {
var found = mimeModes[spec.name];
if (typeof found == "string") { found = {name: found}; }
spec = createObj(found, spec);
spec.name = found.name;
} else if (typeof spec == "string" && /^[\w\-]+\/[\w\-]+\+xml$/.test(spec)) {
return resolveMode("application/xml")
} else if (typeof spec == "string" && /^[\w\-]+\/[\w\-]+\+json$/.test(spec)) {
return resolveMode("application/json")
}
if (typeof spec == "string") { return {name: spec} }
else { return spec || {name: "null"} }
}
```

```
function getMode(options, spec) {
spec = resolveMode(spec);
var mfactory = modes[spec.name];
if (!mfactory) { return getMode(options, "text/plain") }
var modeObj = mfactory(options, spec);
if (modeExtensions.hasOwnProperty(spec.name)) {
var exts = modeExtensions[spec.name];
for (var prop in exts) {
if (!exts.hasOwnProperty(prop)) { continue }
if (modeObj.hasOwnProperty(prop)) { modeObj["_" + prop] = modeObj[prop]; }
modeObj[prop] = exts[prop];
}
}
modeObj.name = spec.name;
if (spec.helperType) { modeObj.helperType = spec.helperType; }
if (spec.modeProps) { for (var prop$1 in spec.modeProps)
{ modeObj[prop$1] = spec.modeProps[prop$1]; } }
return modeObj
}
var modeExtensions = {};
function extendMode(mode, properties) {
var exts = modeExtensions.hasOwnProperty(mode) ? modeExtensions[mode] :
(modeExtensions[mode] = {});
copyObj(properties, exts);
}
function copyState(mode, state) {
if (state === true) { return state }
if (mode.copyState) { return mode.copyState(state) }
var nstate = {};
for (var n in state) {
var val = state[n];
if (val instanceof Array) { val = val.concat([]); }
nstate[n] = val;
}
return nstate
}
function innerMode(mode, state) {
var info;
while (mode.innerMode) {
info = mode.innerMode(state);
if (!info || info.mode == mode) { break }
state = info.state;
mode = info.mode;
}
return info || {mode: mode, state: state}
}
function startState(mode, a1, a2) {
return mode.startState ? mode.startState(a1, a2) : true
}
var StringStream = function(string, tabSize, lineOracle) {
```

```
this.pos = this.start = 0;
this.string = string;
this.tabSize = tabSize || 8;
this.lastColumnPos = this.lastColumnValue = 0;
this.lineStart = 0;
this.lineOracle = lineOracle;
};
StringStream.prototype.eol = function () {return this.pos >=
this.string.length};
StringStream.prototype.sol = function () {return this.pos == this.lineStart};
StringStream.prototype.peek = function () {return this.string.charAt(this.pos)
|| undefined};
StringStream.prototype.next = function () {
if (this.pos < this.string.length)
{ return this.string.charAt(this.pos++) }
};
StringStream.prototype.eat = function (match) {
var ch = this.string.charAt(this.pos);
var ok;
if (typeof match == "string") { ok = ch == match; }
else { ok = ch && (match.test ? match.test(ch) : match(ch)); }
if (ok) {++this.pos; return ch}
};
StringStream.prototype.eatWhile = function (match) {
var start = this.pos;
while (this.eat(match)){}
return this.pos > start
};
StringStream.prototype.eatSpace = function () {
var start = this.pos;
while (/[\s\u00a0]/.test(this.string.charAt(this.pos))) { ++this.pos; }
return this.pos > start
};
StringStream.prototype.skipToEnd = function () {this.pos =
this.string.length;};
StringStream.prototype.skipTo = function (ch) {
var found = this.string.indexOf(ch, this.pos);
if (found > -1) {this.pos = found; return true}
};
StringStream.prototype.backUp = function (n) {this.pos -= n;};
StringStream.prototype.column = function () {
if (this.lastColumnPos < this.start) {
this.lastColumnValue = countColumn(this.string, this.start, this.tabSize,
this.lastColumnPos, this.lastColumnValue);
this.lastColumnPos = this.start;
}
return this.lastColumnValue - (this.lineStart ? countColumn(this.string,
this.lineStart, this.tabSize) : 0)
};
StringStream.prototype.indentation = function () {
```

```
return countColumn(this.string, null, this.tabSize) -
(this.lineStart ? countColumn(this.string, this.lineStart, this.tabSize) : 0)
};
StringStream.prototype.match = function (pattern, consume, caseInsensitive) {
if (typeof pattern == "string") {
var cased = function (str) { return caseInsensitive ? str.toLowerCase() : str; };
var substr = this.string.substr(this.pos, pattern.length);
if (cased(substr) == cased(pattern)) {
if (consume !== false) { this.pos += pattern.length; }
return true
}
} else {
var match = this.string.slice(this.pos).match(pattern);
if (match && match.index > 0) { return null }
if (match && consume !== false) { this.pos += match[0].length; }
return match
}
};
StringStream.prototype.current = function (){return
this.string.slice(this.start, this.pos)};
StringStream.prototype.hideFirstChars = function (n, inner) {
this.lineStart += n;
try { return inner() }
finally { this.lineStart -= n; }
};
StringStream.prototype.lookAhead = function (n) {
var oracle = this.lineOracle;
return oracle && oracle.lookAhead(n)
};
StringStream.prototype.baseToken = function () {
var oracle = this.lineOracle;
return oracle && oracle.baseToken(this.pos)
};
function getLine(doc, n) {
n -= doc.first;
if (n < 0 || n >= doc.size) { throw new Error("There is no line " + (n + doc.first)
+ " in the document.") }
var chunk = doc;
while (!chunk.lines) {
for (var i = 0;; ++i) {
var child = chunk.children[i], sz = child.chunkSize();
if (n < sz) { chunk = child; break }
n -= sz;
}
}
return chunk.lines[n]
}
function getBetween(doc, start, end) {
var out = [], n = start.line;
doc.iter(start.line, end.line + 1, function (line) {
```

```
var text = line.text;
if (n == end.line) { text = text.slice(0, end.ch); }
if (n == start.line) { text = text.slice(start.ch); }
out.push(text);
++n;
});
return out
}
```